ECE414 – Milestone 3

James Ogle

November 12, 2026

Project Progress

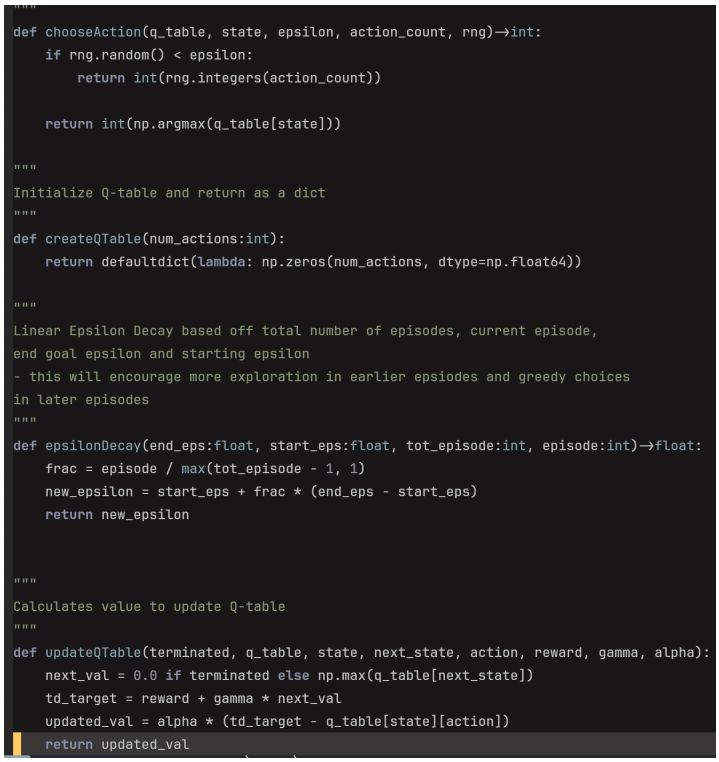
- The focus during this section of the project has been focused developing the Q-learning algorithm for create a Q-table for the agent. After Q-learning was developed, the next part of the process of analyzing the results to confirm the agent was able to "learn" in the environment and improves its actions.
- No major changes have been made to the environment. The only change implemented into the environment was the viewable-space by the agent, changed from 5x5 to 3x3 grid space in front.
- The next focus will be implementing a way for the agent to operate consistently well on grids it has not seen before. Currently, it is able to learn and be 100% accurate the grid it was trained on which does not mean anything for the real world.

RL Algorithm Implementation

- **Algorithm:** Q-learning with linear epsilon decay is the algorithm implemented to train the agent.
- **Reason:** I chose Q-learning since it learns strictly from the policy. This means in ideal sense, it should aim for choosing the most optimal route. Since the environment I have setup does not have meaning hazards to account for, the potential of Q-learning causing rover to fail catastrophically in unseen grids seemed unlikely. However, in order to make sure the agent explores, I implemented linear decay so the agent would explore more in the earlier episodes and becoming more greedy in the later episodes.
- **Action Selection:** The agent uses an epsilon-greedy action-selection policy. With probability ϵ, it selects one of the four movement actions randomly to explore the environment. Otherwise, it selects the action with the largest current Q-value for the observed state. During training, ϵ gradually decreases so the rover explores more at the beginning and relies more on learned actions later.
- **Learning:**

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right].$$

  Here, α is the learning rate and γ is the discount factor. The algorithm directly updates Q-values rather than a separate policy table; the policy improves indirectly because the agent chooses the action with the highest Q-value in each state.



```
"""
def chooseAction(q_table, state, epsilon, action_count, rng)→int:
    if rng.random() < epsilon:
        return int(rng.integers(action_count))

    return int(np.argmax(q_table[state]))

"""
Initialize Q-table and return as a dict
"""
def createQTable(num_actions:int):
    return defaultdict(lambda: np.zeros(num_actions, dtype=np.float64))

"""
Linear Epsilon Decay based off total number of episodes, current episode,
end goal epsilon and starting epsilon
- this will encourage more exploration in earlier epsiodes and greedy choices
in later episodes
"""
def epsilonDecay(end_eps:float, start_eps:float, tot_episode:int, episode:int)→float:
    frac = episode / max(tot_episode - 1, 1)
    new_epsilon = start_eps + frac * (end_eps - start_eps)
    return new_epsilon

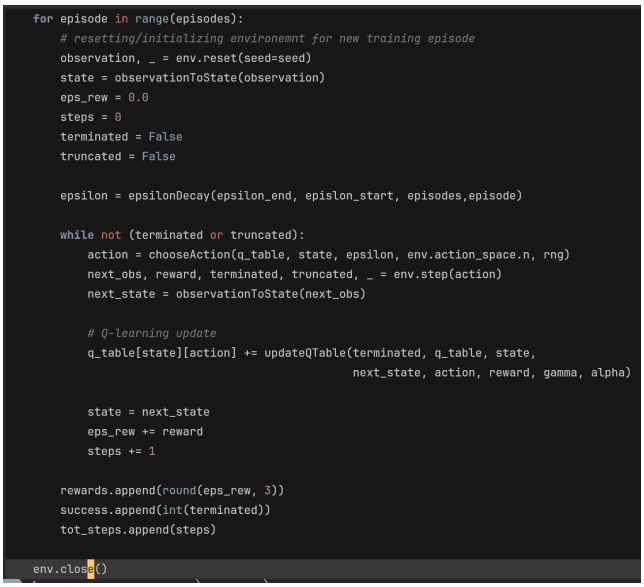

"""
Calculates value to update Q-table
"""
def updateQTable(terminated, q_table, state, next_state, action, reward, gamma, alpha):
    next_val = 0.0 if terminated else np.max(q_table[next_state])
    td_target = reward + gamma * next_val
    updated_val = alpha * (td_target - q_table[state][action])
    return updated_val
```

Figure 1: Code implementation for Reinforcement Learning Algorithm Q-learning

Training Process

- Algorithm implemented for the training process is Q-learning with linear decay for the epsilon
- Parameters used for the algorithm are epsilon start and end, episodes (these 3 are for calculating epsilon decay), discount factor (gamma), and learning rate (alpha)
- The agents Q-values are updated through:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right].$$



```
for episode in range(episodes):
    # resetting/initializing environemnt for new training episode
    observation, _ = env.reset(seed=seed)
    state = observationToState(observation)
    eps_rew = 0.0
    steps = 0
    terminated = False
    truncated = False

    epsilon = epsilonDecay(epsilon_end, epislon_start, episodes,episode)

    while not (terminated or truncated):
        action = chooseAction(q_table, state, epsilon, env.action_space.n, rng)
        next_obs, reward, terminated, truncated, _ = env.step(action)
        next_state = observationToState(next_obs)

        # Q-learning update
        q_table[state][action] += updateQTable(terminated, q_table, state,
                                               next_state, action, reward, gamma, alpha)

        state = next_state
        eps_rew += reward
        steps += 1

    rewards.append(round(eps_rew, 3))
    success.append(int(terminated))
    tot_steps.append(steps)

env.close()
```

Figure 2: Code for training the RL agent

The rover is trained using tabular Q-learning, which updates Q-values that estimate the long-term reward of each action in a given state. At the start of each episode, the environment resets and provides the rover with an initial partial observation. The rover converts this observation into a state and uses an epsilon-greedy policy to select one of four movement actions. After each action, the environment returns the next observation, reward, and episode status, which the agent uses to update the Q-value for its previous state-action pair. In order to update the Q-value, the training process includes epsilon, learning rate, and discount factor into the Q-value update equation. This cycle continues until the rover reaches the goal or the episode reaches the maximum step limit.

Initial Results & Baseline

- The agent is able to achieve a reward value of 1.54 and 19 steps per an episode after training. However this is only when using the same grid for the evaluation. When the grid changes on each episode randomly, then the agent will fail or succeed at random.
- The agent is improving during training which can be seen in the graphs below where the rewards per episode increases as the number of episodes increases. This can also be seen in the steps as well where the number of steps is decreasing over time. The steps decreasing is one of the goals of the agent since it means it is taking less steps per an episode in order to find the goal (aka, it is finding a quicker path each time).
- Currently, the evaluation metrics being used to measure the agent's progress are the number of steps and total reward value per an episode. With the introduction of new environments for the agent each episode, I will introduce a success rate metric to capture how many successful runs it has. I found this metric not to be helpful currently since the agent has 100% success rate during the evaluation but collapses to around 50% when new grids are introduced.

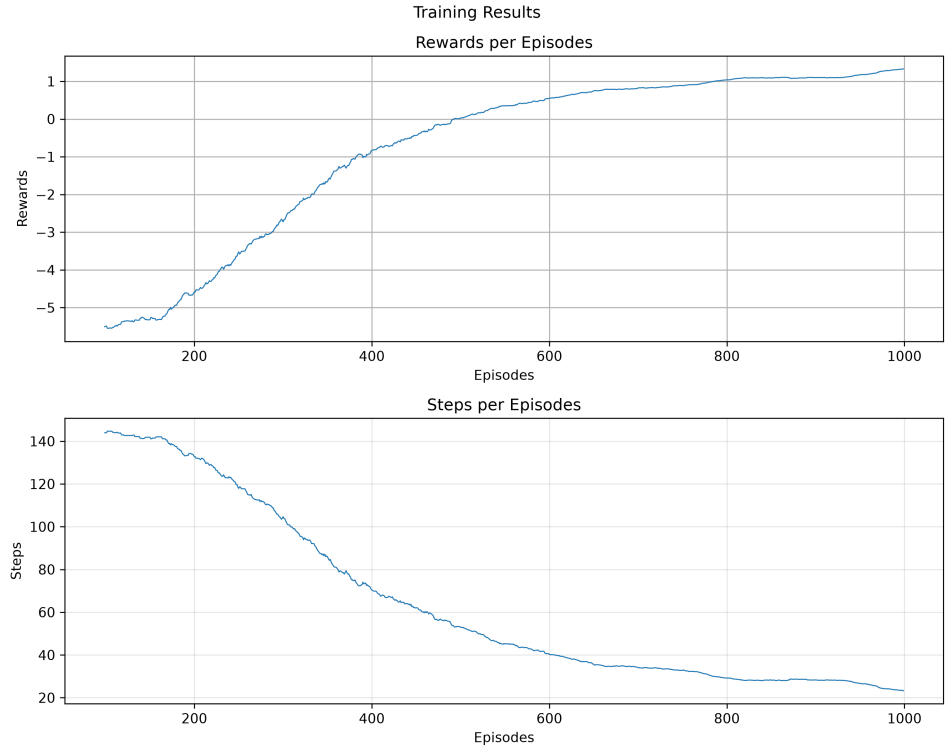


Figure 3: Training Result of 1000 episodes; top graph is reward per episode and bottom is steps per episode

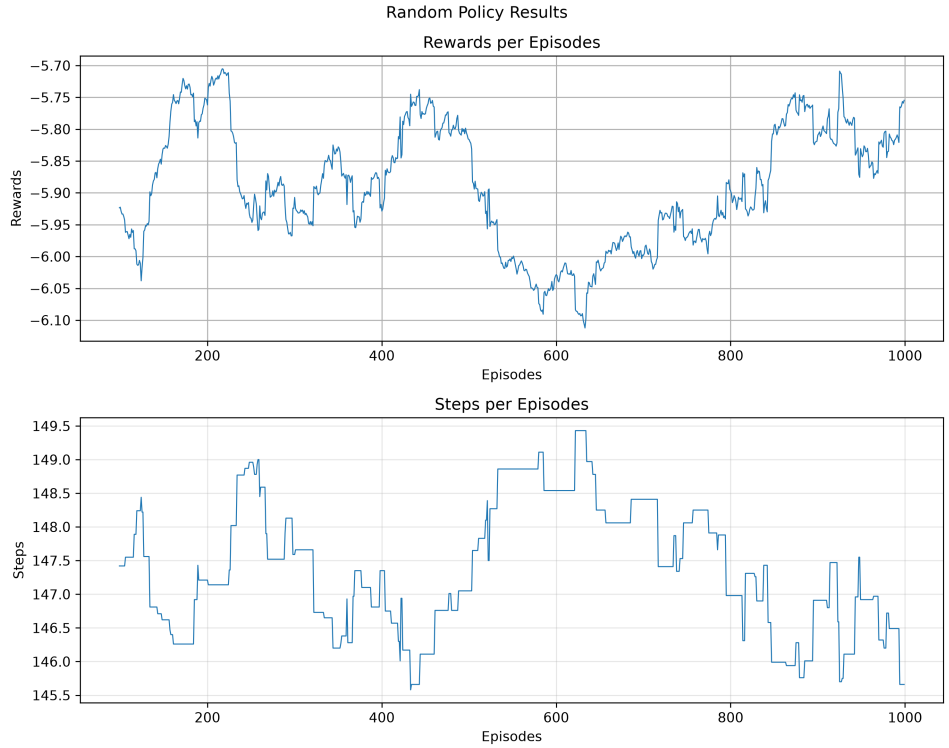


Figure 4: Result of implementing random policy for the agent and running it for 1000 episodes on the same grid

For figure 3, this displays how when implementing Q-learning with linear-decay, the agent improves over each episode. This can be seen by rewards increasing to 1.54 and the steps decreasing to 19 steps. Figure 4 represents the baseline of how the agent performs on the same grid for 1000 episodes. This shows through the training process, we are able to train an agent to discover the end goal of a grid. A demonstration video

of the agent traversing the grid after training is in the zip file listed as `demonstration_milestone3.mp4`

Next Steps

Current problems in the reinforcement agent involves the agent performing well in grids it was not trained on. This can be seen when a seed is not used inside the environment during evaluation, meaning a new grid is generated during each episode. The agent can sometimes find a path to the main objective, though it will not always be the shortest path. A goal for improving the model will be for the agent to achieve 90% accuracy or greater during the evaluation loop with new grids being generated every episode.